Zaawansowane techniki programowania

Laboratorium 9

ZMIENDAK WIKTOR

ZADANIA OBOWIĄZKOWE:

1. OrderServiceImpl używa teraz tylko ProductClient.

```java
public OrderServiceImpl(ProductClient productClient, OrderStore orderStore, Clock clock) {
  this.productClient = productClient;
  this.orderStore = orderStore;
  this.clock = clock;
}

@Override
public Order createOrder(long productId, int quantity) {
  if (quantity <= 0) {
    throw new IllegalArgumentException(s: "Quantity must be > 0");
  }

  var product = productClient.getProduct(productId);
  productClient.reserveProduct(productId, quantity);

  var total = product.price().multiply(java.math.BigDecimal.valueOf(quantity));
  return orderStore.create(product.id(), product.name(), quantity, total, java.time.Instant.now(clock));
}
```

2. Klasy ProductDetails, ProductClient oraz ReserveProductCommand istnieją

3. W ProductServiceImpl dokończono tworzenie, pobieranie, listowanie i rezerwowanie produktów

```java
public ProductServiceImpl(ProductStore store) {
  this.store = store;
}

@Override
public ProductDetails createProduct(CreateProductRequest request) {
  Product product = store.create(request.getName(), request.getPrice(), request.getStock());
  return toDetails(product);
}

@Override
public ProductDetails getProduct(long id) {
  return store.findById(id)
      .map(this::toDetails)
      .orElseThrow(() -> new ProductNotFoundException(message: "Product not found"));
}

@Override
public List<ProductDetails> getProducts() {
  return store.findAll().stream()
      .map(this::toDetails)
      .toList();
}

@Override
public ProductDetails reserveProduct(long id, int quantity) {
  if (quantity <= 0) {
    throw new IllegalArgumentException(s: "Quantity must be > 0");
  }

  Product product = store.findById(id)
      .orElseThrow(() -> new ProductNotFoundException(message: "Product not found"));

  if (product.getStock() < quantity) {
    throw new InsufficientStockException(message: "Insufficient stock");
  }

  product.decreaseStock(quantity);
  return toDetails(product);
}
```

4. W OrderServiceImpl uzupełniono tworzenie zamówienia przy pomocy ProductClient. Jeżeli quantity <= 0 to wyżucany jest błąd.

```
public OrderServiceImpl(ProductClient productClient, OrderStore orderStore, Clock clock) {
  this.productClient = productClient;
  this.orderStore = orderStore;
  this.clock = clock;
}

@Override
public Order createOrder(long productId, int quantity) {
  if (quantity <= 0) {
    throw new IllegalArgumentException(s: "Quantity must be > 0");
  }

  var product = productClient.getProduct(productId);
  productClient.reserveProduct(productId, quantity);

  var total = product.price().multiply(java.math.BigDecimal.valueOf(quantity));
  return orderStore.create(product.id(), product.name(), quantity, total, java.time.Instant.now(clock));
}
```

ZADANIA DODATKOWE:

5. Post/order-service/orders obsługuje teraz OrderController i przy poprawnym requescie wywołuje createOrder.

```
@Component
public class OrderStore {

  private final AtomicLong sequence = new AtomicLong(initialValue: 1);
  private final ConcurrentMap<Long, Order> orders = new ConcurrentHashMap<>();

  public synchronized Order create(Long productId, String productName, int quantity, BigDecimal total, Instant createdAt) {
    Long id = sequence.getAndIncrement();
    Order order = new Order(id, productId, productName, quantity, total, createdAt);
    orders.put(id, order);
    return order;
  }
```

6. **i 7.** Błędne requesty powinny zwracać 400. Brak produktu z klienta daje 404 a brak stanu daje 409.

```
@RestControllerAdvice
public class ApiExceptionHandler {

  @ExceptionHandler({ProductNotFoundException.class, OrderNotFoundException.class})
  @ResponseStatus(HttpStatus.NOT_FOUND)
  public Map<String, String> notFound(RuntimeException ex) {
    return Map.of(k1: "error", ex.getMessage());
  }

  @ExceptionHandler(InsufficientStockException.class)
  @ResponseStatus(HttpStatus.CONFLICT)
  public Map<String, String> conflict(RuntimeException ex) {
    return Map.of(k1: "error", ex.getMessage());
  }

  @ExceptionHandler(IllegalArgumentException.class)
  @ResponseStatus(HttpStatus.BAD_REQUEST)
  public Map<String, String> badRequest(RuntimeException ex) {
    return Map.of(k1: "error", ex.getMessage());
  }

  @ExceptionHandler(MethodArgumentNotValidException.class)
  @ResponseStatus(HttpStatus.BAD_REQUEST)
  public Map<String, String> validation(MethodArgumentNotValidException ex) {
    String message = ex.getBindingResult().getFieldErrors().stream()
        .findFirst()
        .map(error -> error.getField() + ": " + error.getDefaultMessage())
        .orElse(other: "Validation error");
    return Map.of(k1: "error", message);
  }
}
```

8. Jeżeli rezerwacja rzuci wyjątek to utworzenie zamówienia się nie powiedzie.

```java
@Service
public class OrderServiceImpl implements OrderService {

  private final ProductClient productClient;
  private final OrderStore orderStore;
  private final Clock clock;

  public OrderServiceImpl(ProductClient productClient, OrderStore orderStore, Clock clock) {
    this.productClient = productClient;
    this.orderStore = orderStore;
    this.clock = clock;
  }

  @Override
  public Order createOrder(long productId, int quantity) {
    if (quantity <= 0) {
      throw new IllegalArgumentException(s: "Quantity must be > 0");
    }

    var product = productClient.getProduct(productId);
    productClient.reserveProduct(productId, quantity);

    var total = product.price().multiply(java.math.BigDecimal.valueOf(quantity));
    return orderStore.create(product.id(), product.name(), quantity, total, java.time.Instant.now(clock));
  }

  @Override
  public Order getOrder(long id) {
    return orderStore.findById(id)
        .orElseThrow(() -> new OrderNotFoundException(message: "Order not found"));
  }

  @Override
  public List<Order> getOrders() {
    return orderStore.findAll();
  }
}
```

9.
```java
@RestController
@RequestMapping("/order-service/orders")
public class OrdersController {

  private final OrderService orderService;

  public OrdersController(OrderService orderService) {
    this.orderService = orderService;
  }

  @PostMapping
  public ResponseEntity<Order> createOrder(@Valid @RequestBody CreateOrderRequest request) {
    Order created = orderService.createOrder(request.getProductId(), request.getQuantity());
    return ResponseEntity.status(HttpStatus.CREATED).body(created);
  }

  @GetMapping("/{id}")
  public Order getOrder(@PathVariable long id) {
    return orderService.getOrder(id);
  }

  @GetMapping
  public List<Order> getOrders() {
    return orderService.getOrders();
  }
}
```

WYNIKI TESTÓW

```
PROBLEMS  OUTPUT  TERMINAL  PORTS  DEBUG CONSOLE  ROBOT DOCUMENTATION

2026-05-11T18:40:25.643+02:00  INFO 11336 --- [lab9-microservices-starter] [           main] p.e.l.Grade3ProductServiceContractTest
active profile set, falling back to 1 default profile: "default"
2026-05-11T18:40:25.972+02:00  INFO 11336 --- [lab9-microservices-starter] [           main] o.s.b.t.m.w.SpringBootMockServletContex
tializing Spring TestDispatcherServlet ''
2026-05-11T18:40:25.973+02:00  INFO 11336 --- [lab9-microservices-starter] [           main] o.s.t.web.servlet.TestDispatcherServlet
tializing Servlet ''
2026-05-11T18:40:25.973+02:00  INFO 11336 --- [lab9-microservices-starter] [           main] o.s.t.web.servlet.TestDispatcherServlet
pleted initialization in 0 ms
2026-05-11T18:40:25.980+02:00  INFO 11336 --- [lab9-microservices-starter] [           main] p.e.l.Grade3ProductServiceContractTest
rted Grade3ProductServiceContractTest in 0.399 seconds (process running for 8.615)
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 6.060 s -- in pl.example.lab9.Grade3ProductServiceContractTes
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  11.541 s
[INFO] Finished at: 2026-05-11T18:40:26+02:00
[INFO] ------------------------------------------------------------------------
PS C:\Users\ziggs\Desktop\Studies\Cybersecurity\Semestr_I\ZTP\Zmiendak_Wiktor_LAB9\Zmiendak_Wiktor_LAB9>
```

```
PROBLEMS  OUTPUT  TERMINAL  PORTS  DEBUG CONSOLE  ROBOT DOCUMENTATION

2026-05-11T18:41:09.990+02:00  INFO 20472 --- [lab9-microservices-starter] [           main] o.s.b.t.m.w.SpringBootMockSe
tializing Spring TestDispatcherServlet ''
2026-05-11T18:41:09.990+02:00  INFO 20472 --- [lab9-microservices-starter] [           main] o.s.t.web.servlet.TestDispat
tializing Servlet ''
2026-05-11T18:41:09.992+02:00  INFO 20472 --- [lab9-microservices-starter] [           main] o.s.t.web.servlet.TestDispat
pleted initialization in 2 ms
2026-05-11T18:41:10.001+02:00  INFO 20472 --- [lab9-microservices-starter] [           main] p.e.lab9.Grade4OrderApiAndEr
rted Grade4OrderApiAndErrorsTest in 0.337 seconds (process running for 16.555)
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 15.81 s -- in pl.example.lab9.Grade4OrderApiAndErr
[INFO] Running pl.example.lab9.Grade4OrderConsistencyTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.036 s -- in pl.example.lab9.Grade4OrderConsister
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  23.763 s
[INFO] Finished at: 2026-05-11T18:41:10+02:00
[INFO] ------------------------------------------------------------------------
PS C:\Users\ziggs\Desktop\Studies\Cybersecurity\Semestr_I\ZTP\Zmiendak_Wiktor_LAB9\Zmiendak_Wiktor_LAB9>
```